LAUNCHPAD.DEV

Solution Architecture

Frontend Web Application — Architecture & Deployment

Prepared by: Lucky Twala
Diploma in ICT (Application Development)

11 August 2026

1. Introduction & Scope

This document describes the software architecture of the LaunchPad.dev prototype: a career-acceleration web application for ICT students. The current build is a frontend-only single-page application (SPA) — there is no backend server or database. All "accounts", subscriptions, bookings, and progress data live in the browser's localStorage, which keeps the prototype simple to build, run, and deploy while still demonstrating the full user experience end-to-end.

This is an intentional architectural decision for the prototype stage, not an oversight — Section 8 sets out how the architecture would evolve if the platform moved from a class project to a production service.

2. Architectural Style

- Single-Page Application (SPA) — one HTML shell containing all ten screens; navigation swaps which screen is visible rather than loading new pages.
- Client-side hash routing — the URL fragment (e.g. #cv-builder) drives which screen is active, so the browser's back/forward buttons and direct links both work.
- Layered separation within a single script — presentation (HTML/CSS), application logic (routing, validation, scoring), and data (localStorage) are kept in distinct, clearly-commented sections of the codebase even though they run in one JS file.
- No build step — plain HTML/CSS/JS, so the project can be opened locally or published to GitHub Pages with no compilation pipeline.

3. High-Level Architecture

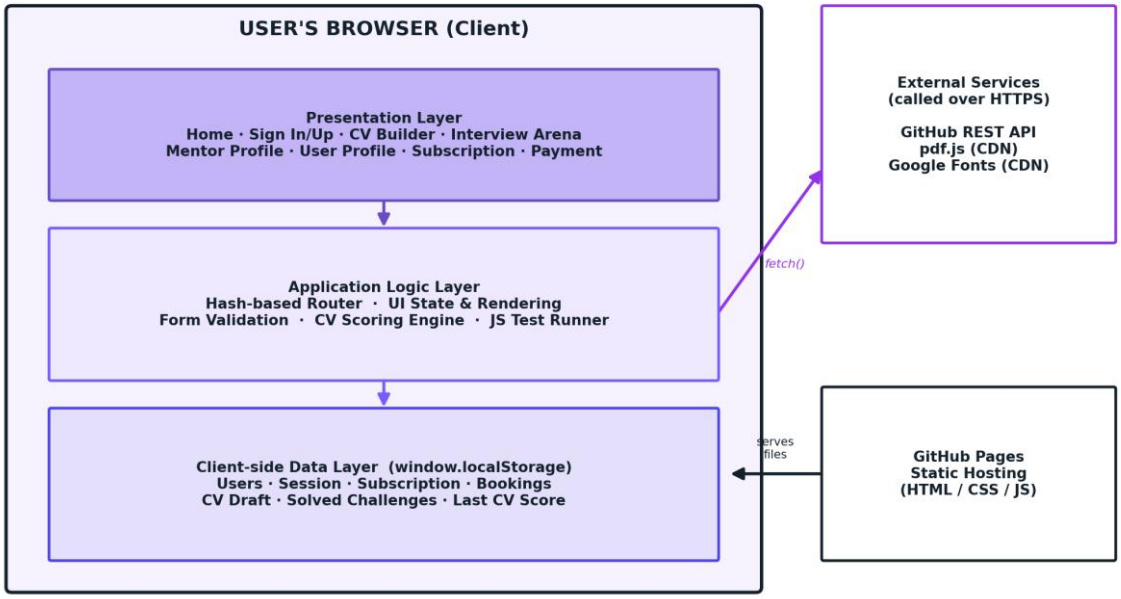


Figure 1 — Layered client architecture, external services, and static hosting.

Everything inside the dashed browser boundary executes entirely on the student's device. The Presentation Layer renders the ten screens; the Application Logic Layer owns routing, form validation, the CV scoring engine, and the in-browser JavaScript test runner used by Interview Arena; the Data Layer wraps all reads/writes to localStorage behind small helper functions (e.g. saveUsers(), saveBookings()) so the rest of the app never touches localStorage directly.

4. Component Breakdown

Screen / Module	Responsibility	Key state
Home	Landing page, mentor discovery, quick links	mentors[] (read)
Sign In / Sign Up / Forgot Password	Local account creation & authentication	lp_users, lp_current_user
CV Builder — Analyze	PDF/text extraction, rule-based CV & GitHub scoring	lp_last_cv_score
CV Builder — Build	Form-driven CV authoring with live preview	lp_cv_builder
Interview Arena	Challenge bank, in-browser JS test execution	lp_solved_challenges
Mentor Profile	Mentor details, reviews, 7-day booking calendar	mentors[].reviews, lp_bookings
User Profile	Editable account info, progress summary	lp_current_user
Subscription	Plan selection, billing cycle toggle	lp_subscription
Payment	Card/Apple Pay validation, plan upgrade	lp_subscription

5. Data Architecture

There is no database. All persistent data is stored client-side as JSON under the keys below. Data is scoped to one browser — it does not sync between devices and is lost if the user clears site data.

localStorage key	Holds
lp_users	Array of locally registered accounts (username, email, password, phone, bio)
lp_current_user	The signed-in user's profile
lp_subscription	Current plan (free/pro) and billing cycle (monthly/annual)
lp_bookings	Array of mentor session bookings (mentor, date, time)
lp_cv_builder	Draft CV built in the "Build From Scratch" tab (autosaved)
lp_solved_challenges	IDs of Interview Arena challenges solved this browser
lp_last_cv_score	Most recent CV audit score, shown on the Profile page

5.1 Key Data Flow — Sign Up / Sign In

- Sign Up validates the form, then appends {username, email, password, phone, bio} to lp_users and writes the new session to lp_current_user.
- Sign In looks up lp_users by email, compares the stored password, and on success writes lp_current_user and flips the nav into the logged-in state.
- Forgot Password looks up the email in lp_users; if found, it lets the user set a new password, which overwrites that record in lp_users.

5.2 Key Data Flow — CV Analysis

- A PDF upload is parsed entirely client-side by pdf.js, extracting raw text per page.
- The extracted (or pasted) text is run through a rule-based scoring function — regex checks for contact details, section headings, word count, and action verbs.
- If a GitHub username/URL is supplied, the app calls the public GitHub REST API directly from the browser to check repo count, bio, and recent activity.
- The combined score is displayed immediately and the latest score is cached in lp_last_cv_score for the Profile page.

6. External Integrations

Service	Purpose	Auth required
GitHub REST API	Fetch public profile & repo data for the CV audit	No (public endpoints, rate-limited)
pdf.js (CDN)	Extract text from an uploaded PDF CV in the browser	No
Google Fonts (CDN)	Loads the Inter and Secular One typefaces	No

7. Deployment Architecture

Because the app has no backend, it can be deployed as static files. GitHub Pages is used to host it directly from the project's GitHub repository.



Figure 2 — Publish flow from a private GitHub repository to a shareable live link.

The repository itself can remain private (so coursework/source code isn't publicly browsable), while GitHub Pages still serves the compiled static site at a shareable URL for demonstration and marking purposes.

8. Technology Stack

Layer	Technology
Markup / structure	HTML5 (single-page shell, 10 screen sections)
Styling	CSS3 — custom properties (design tokens), responsive grid/flexbox
Behaviour	Vanilla JavaScript (ES5-compatible), no framework or build step
Client-side PDF parsing	pdf.js (Mozilla), loaded via CDN
External data	GitHub REST API (public, unauthenticated) for repo/activity checks
Fonts	Google Fonts (Inter, Secular One), loaded via CDN
Persistence	Browser localStorage — no backend or database
Hosting	GitHub Pages (static hosting, HTTPS)

9. Security Considerations & Limitations

- No real authentication: passwords are stored in plain text in localStorage. This is acceptable for a demo but must never be reused as a real password.
- No server-side validation: all checks (email format, Luhn card check, etc.) run in the browser and could be bypassed by a technical user — fine for a prototype, not for production.
- Data is local to one browser: clearing site data, using a different browser, or a different device loses all saved state.
- Payments are simulated: no real payment gateway is integrated; card details are validated for format only and never transmitted anywhere.
- GitHub API calls are unauthenticated and therefore rate-limited (60 requests/hour per IP) — acceptable for occasional student use, not for scale.

10. Future Architecture (Beyond Prototype)

- Introduce a backend (e.g. Node/Express or Spring Boot) with a real database (PostgreSQL/MySQL) to replace localStorage.
- Add proper authentication with hashed passwords (e.g. bcrypt) and session tokens (JWT) instead of storing plaintext credentials client-side.
- Integrate a real payment gateway (e.g. PayFast or Stripe) for subscription billing.
- Move hosting from GitHub Pages to a platform that supports a backend (e.g. Render, Railway, or Azure) once server-side logic is introduced.
- Add authenticated GitHub OAuth so CV audits can read private repo activity with the student's consent, rather than public data only.